1. Implementation of Authentication

- **Code Snippets**

Interface:

```
namespace GharBazar.API.Services;

public interface IAuthService
{
    Task<(bool Success, string? Token, string? Error)> LoginAsync(string email, string password);
    Task<(bool Success, string? UserId, string? Otp, string? Error)> RegisterAsync(string userName, string email, string password, string fullName, string role);
    string HashPassword(string password);
    bool VerifyPassword(string password, string hash);
    string GenerateJwtToken(User user, string secretKey, string issuer, string audience);
}

public class AuthService : IAuthService
{
    private readonly IUserRepository _userRepository;

    public AuthService(IUserRepository userRepository)
    {
        _userRepository = userRepository;
    }

    public async Task<(bool Success, string? Token, string? Error)> LoginAsync(string email, string password)
    {
        var user = await _userRepository.GetByEmailAsync(email);
        if (user == null || !user.IsActive)
        {
            return (false, null, "Invalid email or password");
        }

        if (!user.IsEmailVerified)
        {
            return (false, null, "Please verify your email address to continue. Check your inbox for the OTP.");
        }

        var isPasswordValid = VerifyPassword(password, user.PasswordHash);
```

Controller:

```
namespace GharBazar.API.Controllers;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly IAuthService _authService;
    private readonly IUserRepository _userRepository;
    private readonly IConfiguration _configuration;
    private readonly IEmailService _emailService;
    private readonly INotificationRepository _notificationRepository;

    public AuthController(
        IAuthService authService,
        IUserRepository userRepository,
        IConfiguration configuration,
        IEmailService emailService,
        INotificationRepository notificationRepository)
    {
        _authService = authService;
        _userRepository = userRepository;
        _configuration = configuration;
        _emailService = emailService;
        _notificationRepository = notificationRepository;
    }

    [HttpPost("login")]
    public async Task<ActionResult<LoginResponse>> Login([FromBody] LoginRequest request)
    {
        if (string.IsNullOrWhiteSpace(request.Email) || string.IsNullOrWhiteSpace(request.Password))
        {
            return BadRequest(new { message = "Email and password are required" });
        }

        var (success, userId, error) = await _authService.LoginAsync(request.Email, request.Password);
        if (!success)
```

```
public async Task<ActionResult<LoginResponse>> Login([FromBody] LoginRequest request)
{
    if (string.IsNullOrWhiteSpace(request.Email) || string.IsNullOrWhiteSpace(request.Password))
    {
        return BadRequest(new { message = "Email and password are required" });
    }

    var (success, userId, error) = await _authService.LoginAsync(request.Email, request.Password);
    if (!success)
    {
        return Unauthorized(new { message = error });
    }

    var user = await _userRepository.GetByIdAsync(userId!);
    if (user == null)
    {
        return Unauthorized(new { message = "User not found" });
    }

    var token = _authService.GenerateJwtToken(
        user,
        _configuration["Jwt:SecretKey"] ?? "/BrTJGte0zlNTZxLz5zRFdYrYctx9XKOMX/UFSIV0u4D3WVwJ8UysmNI59VKgEWcTmaX4sSAxRG9Jc2BTNIVZw==",
        _configuration["Jwt:Issuer"] ?? "GharBazar",
        _configuration["Jwt:Audience"] ?? "GharBazarUsers"
    );

    return Ok(new LoginResponse
    {
        UserId = user.UserId,
        Email = user.Email,
        Role = user.Role,
        Token = token
    });
}
```

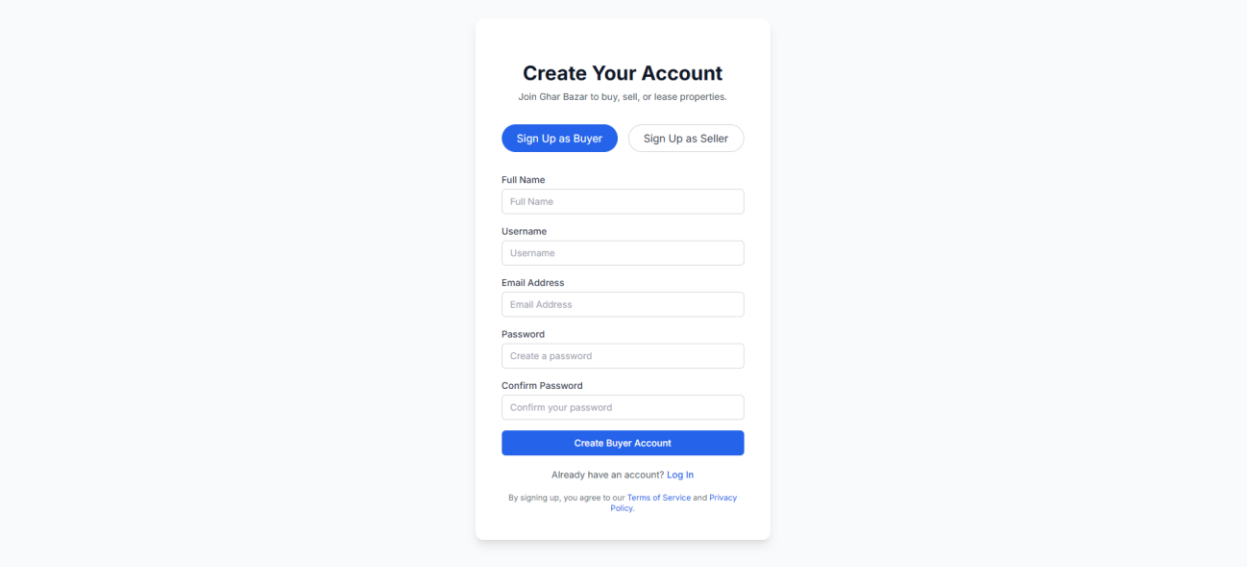
```
public async Task<ActionResult<LoginResponse>> Register([FromBody] RegisterRequest request)
{
    try
    {
        if (string.IsNullOrWhiteSpace(request.Email) || string.IsNullOrWhiteSpace(request.Password) ||
            string.IsNullOrWhiteSpace(request.UserName))
        {
            return BadRequest(new { message = "Email, password, and username are required" });
        }

        var (success, userId, otp, error) = await _authService.RegisterAsync(
            request.UserName,
            request.Email,
            request.Password,
            request.FullName,
            request.Role
        );

        if (!success)
        {
            return BadRequest(new { message = error });
        }

        // Send OTP email
        _ = _emailService.SendEmailAsync(
            request.Email,
            "Verify Your Email Address - GharBazar",
            EmailTemplates.EmailVerificationOtp(otp!)
        );

        return Ok(new { message = "Registration successful. Please check your email for the verification code." });
```

DTOs

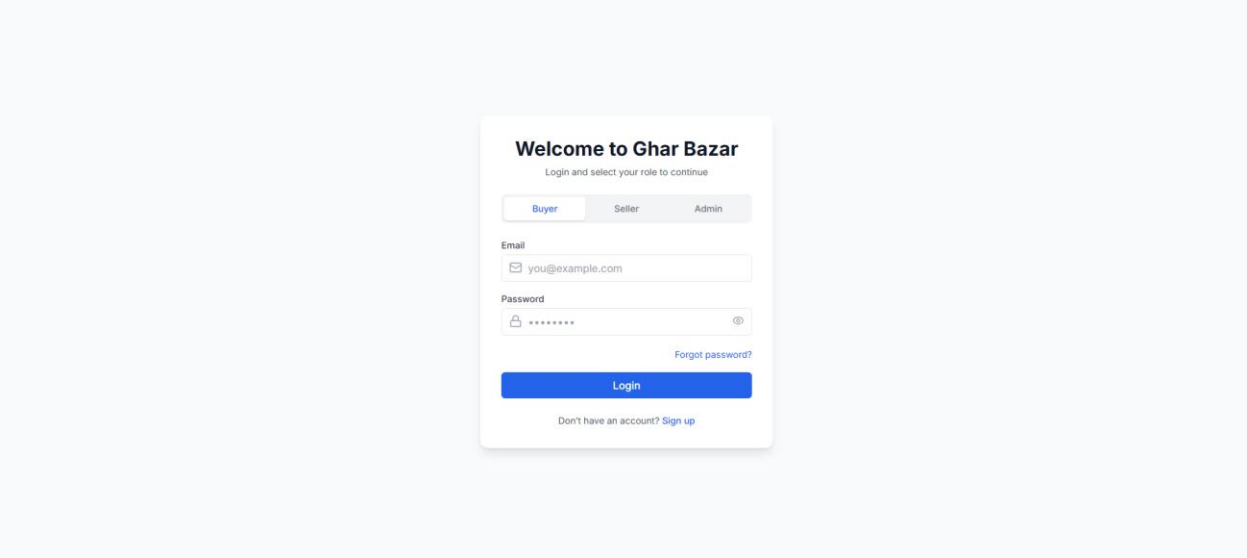
```
// Auth DTOs
public class LoginRequest
{
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}

public class LoginResponse
{
    public string UserId { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
    public string Token { get; set; } = string.Empty;
}

public class RegisterRequest
{
    public string UserName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
    public string FullName { get; set; } = string.Empty;
    public string Role { get; set; } = "BUYER";
    public string? PhoneNumber { get; set; }
}
```

Implementation Proof:

Signup page:



Login Page:



Database Setup:

```
MariaDB [ghar_bazar]> desc users;
+-----------------------------+--------------+------+-----+---------+-------+
| Field                       | Type         | Null | Key | Default | Extra |
+-----------------------------+--------------+------+-----+---------+-------+
| user_id                     | varchar(255) | NO   | PRI | NULL    |       |
| user_name                   | varchar(255) | NO   | UNI | NULL    |       |
| email                       | varchar(255) | NO   | UNI | NULL    |       |
| password_hash               | varchar(255) | NO   |     | NULL    |       |
| role                        | varchar(255) | NO   |     | NULL    |       |
| full_name                   | varchar(255) | YES  |     | NULL    |       |
| phone_number                | varchar(255) | YES  |     | NULL    |       |
| profile_picture_url         | varchar(255) | YES  |     | NULL    |       |
| bio                         | varchar(255) | YES  |     | NULL    |       |
| address                     | varchar(255) | YES  |     | NULL    |       |
| created_at                  | datetime(6)  | NO   |     | NULL    |       |
| updated_at                  | datetime(6)  | NO   |     | NULL    |       |
| is_active                   | tinyint(1)   | NO   |     | 1       |       |
| password_reset_token        | varchar(255) | YES  |     | NULL    |       |
| password_reset_token_expiry | datetime(6)  | YES  |     | NULL    |       |
| email_otp                   | varchar(255) | YES  |     | NULL    |       |
| email_otp_expiry            | datetime(6)  | YES  |     | NULL    |       |
| is_email_verified           | tinyint(1)   | NO   |     | 0       |       |
+-----------------------------+--------------+------+-----+---------+-------+
18 rows in set (0.013 sec)
```